DATA MINING PROJECT REPORT

Stock Trade Analysis of the Oil & Gas Sector in Pakistan

Pakistan Stock Exchange (PSX): 2016 to 2026

Team Members

Abdul Basit (22F-BSAI-25)

Muhammad Hammad (22F-BSAI-39)

Muhammad Sohaib (22F-BSAI-40)

Omaid Ejaz (22F-BSAI-45)

Wajhi-Ur-Rahman (22F-BSAI-50)

Submitted to: Engr. Bushra Sheikh

Table of Contents

Abstract 3
1. Introduction 4
1.1 Background & Motivation 4
1.2 Problem Statement 4
1.3 Objectives 4
2. Dataset Description 5
2.1 Data Source & Companies 5
2.2 Data Attributes 6
2.3 Dataset Overview 6
3. Methodology 7
3.1 Data Preprocessing 7
3.2 Feature Engineering 8
3.3 Feature Selection 9
3.4 Data Transformation & Splitting 9
3.5 Data Splitting & Chronological Validation 10
4. Model Development 11
4.1 Algorithms Benchmarked 11
4.2 Hyperparameter Tuning 12
4.3 Model Selection Rationale 14
5. Results & Evaluation 15
5.1 Baseline Model Performance 15
5.2 Tuned Model Performance 15
5.3 Overfitting Analysis 16
6. Dashboard & Deployment 17
7. Conclusion 23
8. References 24

Abstract

This report presents a comprehensive data mining and machine learning study applied to the Oil and Gas sector of the Pakistan Stock Exchange (PSX). The study spans a decade of daily trading data (2016–2026) encompassing 17 listed companies, amounting to tens of thousands of records. The primary objective is to build an accurate, interpretable model capable of predicting next-day and 15-day future stock prices.

A rigorous pipeline was constructed covering data collection, preprocessing, feature engineering, feature selection, model benchmarking, hyperparameter tuning, and deployment. Thirteen machine learning models from diverse algorithm families were evaluated using chronologically stratified train-test splits to prevent data leakage. The Tuned Random Forest Regressor emerged as the optimal model, achieving a Test R-Squared of 99.84% and a Mean Absolute Percentage Error (MAPE) of 2.21%, demonstrating excellent generalization with minimal overfitting.

The final solution is deployed as an interactive Streamlit web application featuring exploratory data analysis (EDA) dashboards, candlestick charts, 15-day autoregressive forecasts, and a manual prediction module — providing a complete financial intelligence platform for the PSX energy sector.

1. Introduction

1.1 Background & Motivation

The Pakistan Stock Exchange (PSX) is the primary securities market of Pakistan, hosting hundreds of listed companies across multiple sectors. The Oil and Gas sector holds particular economic significance, as it underpins energy production, refining, distribution, and infrastructure across the country. Price movements in this sector are influenced by global crude oil prices, domestic regulatory policy, currency fluctuations, and macroeconomic cycles.

With the rapid advancement of machine learning and the growing availability of historical financial data, data-driven forecasting has become a critical tool for investors, analysts, and policymakers. Accurate short-term price prediction enables better portfolio management, risk mitigation, and strategic investment decisions.

1.2 Problem Statement

Despite the availability of a decade of PSX trading data, there is a lack of a unified, data-driven forecasting framework specifically for the Pakistani Oil and Gas sector. Traditional technical analysis methods are limited in their ability to model complex, non-linear relationships between multiple financial variables. This project addresses that gap by developing an end-to-end machine learning pipeline that can predict next-day stock prices and generate 15-day rolling forecasts for all 17 Oil and Gas companies listed on the PSX.

1.3 Objectives

- Collect and consolidate 10 years of daily stock data for 17 PSX Oil and Gas companies.
- Perform thorough data preprocessing, cleaning, and feature engineering to create meaningful financial indicators.
- Benchmark a comprehensive set of machine learning algorithms across multiple families.
- Identify and deploy the best-performing model after rigorous hyperparameter tuning.
- Build an interactive, user-facing Streamlit dashboard for visualization, exploration, and real-time prediction.
- Generate 15-day autoregressive price forecasts using a rolling-window approach.

2. Dataset Description

2.1 Data Source & Companies

The dataset was compiled from publicly available PSX historical trading records. Daily OHLCV data was scraped and aggregated into a single unified CSV file. The study covers 17 companies spanning refining, exploration, distribution, and gas utilities sub-sectors.

The following 17 companies from the Pakistan Stock Exchange (PSX) are included in the dataset, categorized by their primary business operations:

1. Exploration & Production (E&P)
These companies focus on the discovery and extraction of oil and natural gas.

- **MARI:** Mari Petroleum Company Limited
- **OGDC:** Oil & Gas Development Company Limited
- **POL:** Pakistan Oilfields Limited
- **PPL:** Pakistan Petroleum Limited

2. Refining & Processing
Companies dedicated to refining crude oil into usable fuel and products.

- **ATRL:** Attock Refinery Limited
- **NRL:** National Refinery Limited
- **PRL:** Pakistan Refinery Limited
- **CNERGY:** Cnergyico PK Limited

3. Oil Marketing & Distribution
The consumer-facing side of the industry, managing gas stations and fuel supply chains.

- **APL:** Attock Petroleum Limited
- **HASCOL:** Hascol Petroleum Limited
- **PSO:** Pakistan State Oil Company Limited
- **HTL:** Hi-Tech Lubricants Limited
- **WAFI:** Wafi Energy Pakistan Limited
- **OBOY:** Oilboy Energy Limited

4. Gas Utilities & LPG
Entities responsible for the transmission, distribution, and sale of natural gas and LPG.

- **SSGC:** Sui Southern Gas Company Limited
- **SNGP:** Sui Northern Gas Pipelines Limited
- **BLPL:** Burshane LPG (Pakistan) Ltd.

2.2 Data Attributes

Each daily record contains the following raw attributes:

- **Company & Symbol** — Company name and PSX ticker code.
- **Date** — Trading date (business days only, excluding weekends and public holidays).
- **Price (Close)** — The closing price of the stock in Pakistani Rupees (PKR).
- **Open** — The opening price at the start of the trading session.
- **High** — The highest traded price during the session.
- **Low** — The lowest traded price during the session.
- **Volume** — Total number of shares traded, originally formatted as string shorthand (e.g., 1.5M, 200K).
- **Change %** — Percentage change from the previous session's closing price.

2.3 Dataset Overview

	Company	Symbol	Date	Price	Open	High	Low	Vol.	Change %	Country
1812	Attock Petroleum	APL	2023-05-02 00:00:00	303.77	301.80	306.00	297.55	147.63K	0.0092	Pakistan
36020	Sui Southern Gas Co	SSGC	2019-02-07 00:00:00	25.17	25.50	25.75	25.10	857.00K	-0.0091	Pakistan
5491	Burshane LPG (Pakistan)	BLPL	2017-09-14 00:00:00	49.50	48.99	49.65	48.50	15.50K	0.0272	Pakistan
36773	Sui Southern Gas Co	SSGC	2022-02-18 00:00:00	10.41	9.60	10.45	9.54	4.11M	0.0958	Pakistan
32695	Pakistan State Oil	PSO	2026-04-01 00:00:00	348.82	336.00	355.20	333.00	4.28M	0.0612	Pakistan
30473	Pakistan State Oil	PSO	2017-04-10 00:00:00	241.72	240.45	244.21	239.01	192.50K	0.0017	Pakistan
12194	Hi Tech Lubricants	HTL	2016-03-10 00:00:00	49.12	48.58	49.72	48.38	1.15M	0.0155	Pakistan
27472	Pakistan Petroleum	PPL	2025-09-26 00:00:00	208.82	205.00	212.35	202.51	17.78M	0.0271	Pakistan
14791	Mari Petroleum Company	MARI	2017-11-12 00:00:00	1314.05	1322.31	1332.23	1280.99	45.91K	-0.0062	Pakistan
37784	Sui Southern Gas Co	SSGC	2026-03-24 00:00:00	21.94	23.25	23.40	21.80	2.89M	-0.0313	Pakistan
19118	Oil and Gas Development Co	OGDC	2020-11-10 00:00:00	97.75	97.50	99.40	97.10	6.61M	0.0403	Pakistan
21945	Oilboy Energy	OBOY	2023-12-22 00:00:00	6.44	6.49	6.99	6.40	70.00K	-0.0316	Pakistan
28485	Pakistan Refinery	PRL	2019-07-16 00:00:00	11.23	11.02	11.23	10.79	92.55K	0.0108	Pakistan
17973	Oil and Gas Development Co	OGDC	2025-07-02 00:00:00	222.92	222.45	223.99	221.31	4.66M	0.0061	Pakistan
1853	Attock Petroleum	APL	2023-07-03 00:00:00	311.20	314.14	318.00	308.00	185.27K	0.0339	Pakistan

3. Methodology

3.1 Data Preprocessing

Raw data required extensive cleaning prior to analysis. The key preprocessing steps undertaken were:

1. **Volume Conversion:** The Volume column was stored as abbreviated string values (e.g., '1.5M', '500K'). A custom parsing function was written to convert these into numeric integer values for mathematical operations.

```python
# --- 1.5 PRE-PROCESSING: Clean Volume Column ---
# Rename 'Vol.' to 'Volume'
if 'Vol.' in df.columns:
    df.rename(columns={'Vol.': 'Volume'}, inplace=True)

# Convert strings like '0.75K' or '1.3M' to actual numbers
def convert_volume(val):
    if pd.isna(val):
        return 0.0
    val_str = str(val).strip().upper()
    if val_str.endswith('K'):
        return float(val_str[:-1]) * 1000
    elif val_str.endswith('M'):
        return float(val_str[:-1]) * 1000000
    elif val_str.endswith('B'):
        return float(val_str[:-1]) * 1000000000
    else:
        try:
            return float(val_str.replace(',', ''))
        except ValueError:
            return 0.0

if 'Volume' in df.columns:
    df['Volume'] = df['Volume'].apply(convert_volume)

print(f"Loaded {len(df)} rows. Starting Feature Engineering...")
```

2. **Missing Value Handling:** Records with null values in essential fields (Price, Open, High, Low) were identified and removed to ensure model integrity. For moving average features, records with insufficient historical lookback were similarly dropped.
3. **Data Type Correction:** String-typed numeric columns were cast to float64. The Date column was parsed into a datetime index and used to extract temporal features.
4. **Duplicate Record Check:** The dataset was scanned for duplicate Date–Symbol combinations and de-duplicated where applicable.

3.2 Feature Engineering

To improve the model's predictive accuracy, raw data was transformed into three primary categories of engineered features:

```python
# 2. Process each company separately to avoid mixing data
all_companies_processed = []

for symbol in df['Symbol'].unique():
    # Get data for one company and sort it by date
    comp_df = df[df['Symbol'] == symbol].sort_values('Date').copy()

    # --- A. LAG FEATURES ---
    # We use .bfill() so the first few days just copy the earliest known price instead of being NaN
    comp_df['Price_Yesterday'] = comp_df['Price'].shift(1).bfill()
    comp_df['Price_5_Days_Ago'] = comp_df['Price'].shift(5).bfill()

    # --- B. MOVING AVERAGES ---
    # min_periods=1 ensures calculation even on day 1
    comp_df['MA7'] = comp_df['Price'].rolling(window=7, min_periods=1).mean()
    comp_df['MA30'] = comp_df['Price'].rolling(window=30, min_periods=1).mean()

    # --- C. DAILY RETURNS ---
    # comp_df['Daily_Return'] = comp_df['Price'].pct_change().fillna(0)

    # --- D. TIME FEATURES ---
    comp_df['Day_Of_Week'] = comp_df['Date'].dt.dayofweek
    comp_df['Month'] = comp_df['Date'].dt.month

    # --- E. THE TARGET COLUMN ---
    comp_df['Target_Next_Day_Price'] = comp_df['Price'].shift(-1)

    all_companies_processed.append(comp_df)

# 3. Combine all companies back together
final_df = pd.concat(all_companies_processed)
```

1. Lag Features

Lag features allow the model to understand immediate historical context by shifting price data from previous time steps.

- **Price_Yesterday:** The closing price from the immediately preceding trading day (1-day lag).
- **Price_5_Days_Ago:** The closing price recorded 5 trading days prior.

2. Moving Average Features

These features smooth out short-term fluctuations to highlight broader price trends.

- **MA7 (Short-term):** A 7-day rolling mean of closing prices, used to identify short-term momentum.
- **MA30 (Medium-term):** A 30-day rolling mean of closing prices, used to capture medium-term market trends.

3. Temporal Features

Temporal features encode time-based patterns and seasonality into numerical values the model can process.

- **Day_Of_Week:** An integer encoding of the specific trading day (where **0 = Monday** and **4 = Friday**).
- **Month:** The calendar month of the trading record, represented numerically from **1 to 12**.

These features capture price momentum, short-term and medium-term trends, and weekly market cycle effects — all of which are well-established predictors in financial time-series modelling.

3.3 Feature Selection

To reduce noise and improve model efficiency, correlation-based feature selection was applied. Each feature's Pearson correlation coefficient with the target variable (Target_Next_Day_Price) was computed. Features with an absolute correlation below 5% (0.05) were eliminated.

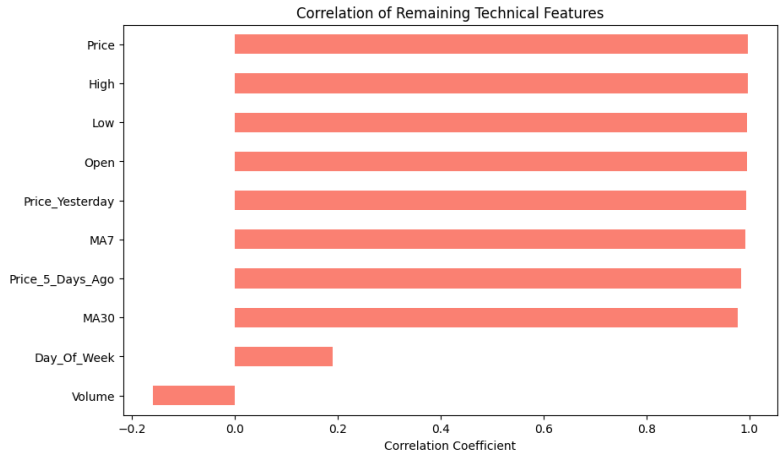


Figure 1

As illustrated in the correlation chart (Figure 1), the top-correlated features were Price, High, Low, Open, Price_Yesterday, MA7, Price_5_Days_Ago, and MA30 all with near-perfect correlation coefficients close to 1.0. Volume showed a negative correlation of approximately -0.17, and Day_Of_Week showed a modest positive correlation of around 0.20. Both were retained as they exceeded the 5% threshold, providing supplementary signal.

3.4 Data Transformation & Splitting

Two transformations were applied before model training:

- **One-Hot Encoding (OHE):** The categorical Symbol column (17 company tickers) was one-hot encoded, producing 17 binary columns. This enables a single multi-company model to learn company-specific price dynamics without assuming ordinal relationships.

```
# --- G. ONE-HOT ENCODING (OHE) ---
print("Applying One-Hot Encoding to Symbols...")
final_df = pd.get_dummies(final_df, columns=['Symbol'], prefix='Symbol')
```

Symbol_APL	Symbol_ATRL	Symbol_BLPL	Symbol_CNERGY	Symbol_HASCOL	Symbol_HTL	Symbol_MARI
True	False	False	False	False	False	False
True	False	False	False	False	False	False
True	False	False	False	False	False	False
True	False	False	False	False	False	False
True	False	False	False	False	False	False

- **Standard Scaling:** All numerical features were normalized using StandardScaler (zero mean, unit variance). This is essential for distance-based models (KNN, SVR) and gradient-based algorithms to converge correctly. The fitted scaler was saved as oil_data_scaler.pkl for use during inference.

```
# --- H. FEATURE SCALING ---
print("Scaling numeric features...")
scaler = StandardScaler()

# List of columns to scale (Updated to 'Volume' instead of 'Vol.')
features_to_scale = [
    'Price', 'Open', 'High', 'Low', 'Volume',
    'Price_Yesterday', 'Price_5_Days_Ago',
    'MA7', 'MA30', 'Change %', 'Day_Of_Week', 'Month'
]

final_df[features_to_scale] = scaler.fit_transform(final_df[features_to_scale])
```

	Price	Open	High	Low	Volume	Change %	Price_Yesterday	Price_5_Days_Ago	MA7	MA30	Day_Of_Week	Month	Target_Next_Day_Price
0	0.808036	0.812886	0.783043	0.837252	-0.375507	-0.453278	0.808517	0.810262	0.812103	0.824198	1.320771	-1.538843	330.61
1	0.799872	0.797108	0.782997	0.821128	-0.374869	-0.191219	0.808517	0.810262	0.808004	0.820052	-0.712054	-1.538843	328.00
2	0.787555	0.797108	0.767606	0.808238	-0.375153	-0.280784	0.800350	0.810262	0.802515	0.814499	-0.034446	-1.538843	324.00
3	0.768680	0.781376	0.752215	0.795349	-0.374657	-0.423423	0.788030	0.810262	0.795032	0.806929	0.643162	-1.538843	320.01
4	0.749851	0.759314	0.730631	0.779322	-0.375259	-0.426740	0.769149	0.810262	0.786761	0.798562	1.320771	-1.538843	319.28

3.5 Data Splitting & Chronological Validation

Data was split into training and test sets using an 80/20 chronological split with shuffle=False. This preserves the temporal ordering of observations, which is critical for time-series forecasting — it prevents data leakage by ensuring the model is never trained on data from dates after those in the test set.

```
from sklearn.model_selection import train_test_split

# Split data: 80% for training (the past) and 20% for testing (the future)
# shuffle=False ensures chronological order is kept intact!
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2, shuffle=False)

print(f"Training data (X) shape: {X_train.shape}")
print(f"Testing data (X) shape: {X_test.shape}")
print(f"Training target (Y) shape: {Y_train.shape}")
print(f"Testing target (Y) shape: {Y_test.shape}")
```

✓ 0.4s

```
Training data (X) shape: (32264, 27)
Testing data (X) shape: (8066, 27)
Training target (Y) shape: (32264,)
Testing target (Y) shape: (8066,)
```

4. Model Development

4.1 Algorithms Benchmarked

Thirteen machine learning algorithms from four major families were evaluated in the baseline comparison phase:

```python
# 1. Define all the models in a dictionary
models = {
    "Linear Regression": LinearRegression(),
    "Ridge Regression": Ridge(),
    "Lasso Regression": Lasso(),
    "Elastic Net": ElasticNet(),
    "Decision Tree": DecisionTreeRegressor(random_state=42),
    "Random Forest": RandomForestRegressor(n_estimators=100, random_state=42),
    "Gradient Boosting (Sklearn)": GradientBoostingRegressor(random_state=42),
    "SVR (Support Vector)": SVR(),
    "K-Nearest Neighbors": KNeighborsRegressor(),
    "AdaBoost": AdaBoostRegressor(random_state=42)
}

if has_xgb:
    models["XGBoost"] = XGBRegressor(random_state=42)
if has_lgbm:
    models["LightGBM"] = LGBMRegressor(random_state=42)
if has_catboost:
    models["CatBoost"] = CatBoostRegressor(verbose=0, random_state=42)


# 2. Train, Predict, Evaluate, and SAVE
results = {}
n = len(Y_test)
p = X_test.shape[1]

print("\n🚀 Training models & Saving to models/ folder...\n" + "="*60)

for name, model in models.items():
    try:
        # Train the model
        model.fit(X_train, Y_train)

        # 💾 SAVE THE MODEL IMMEDIATELY AFTER TRAINING
        # Cleaning the name to create a valid filename (removing spaces/brackets)
        filename = f"models/{name.replace(' ', '_').replace('(', '').replace(')', '').lower()}.pkl"
        joblib.dump(model, filename)

        # Predict on Test Data
        preds = model.predict(X_test)
```

To identify the most accurate forecasting model, several algorithms were evaluated based on their unique architectural characteristics:

- **Linear Regression:** Utilizes ordinary least squares to provide a highly interpretable baseline.
- **Ridge Regression:** Implements **L2 regularization** to reduce coefficient magnitudes and prevent overfitting.
- **Lasso Regression:** Employs **L1 regularization**, which performs implicit feature selection by zeroing out less important variables.
- **Elastic Net:** A hybrid approach that combines both **L1 and L2 penalties** for more robust regularization.

- **Decision Tree:** Uses non-linear splits; provides high detail but is prone to overfitting without proper pruning.
- **Random Forest:** Uses a **bagging** technique with multiple trees to create a robust, low-variance model.
- **AdaBoost:** An adaptive boosting technique that builds sequential weak learners to improve accuracy.
- **Gradient Boosting:** A standard Scikit-learn implementation of boosting that utilizes shrinkage for better generalization.
- **XGBoost:** An optimized gradient boosting library that includes built-in regularization.
- **LightGBM:** Uses a leaf-wise growth strategy, making it exceptionally fast on large datasets.
- **CatBoost:** Utilizes ordered boosting and is specifically designed to handle categorical variables natively.
- **SVR (Support Vector Regression):** Uses an epsilon-insensitive loss function; highly effective in high-dimensional feature spaces.
- **K-Nearest Neighbors (KNN):** An instance-based learner that predicts values based on the average of the local neighborhood.

4.2 Hyperparameter Tuning

After the baseline benchmark, the top-performing candidate models were subjected to systematic hyperparameter optimisation using two strategies:

- **GridSearchCV:** Exhaustive search over a predefined parameter grid. Applied to smaller models (Ridge, Lasso, Elastic Net, Decision Tree, KNN, SVR) where the parameter space is manageable.

```
# --- 1. Decision Tree Tuning ---
# Focus: Controlling tree growth to prevent overfitting
dt_params = {
    'max_depth': [None, 5, 10, 20],
    'min_samples_split': [2, 10, 20],
    'min_samples_leaf': [1, 5, 10]
}
dt_grid = GridSearchCV(DecisionTreeRegressor(random_state=42), dt_params, cv=5)
dt_grid.fit(X_train, Y_train)
tree_results.append(calculate_all_metrics(dt_grid.best_estimator_, X_train, X_test, Y_train, Y_test, 'Decision Tree Tuned'))
# save decision tree model
with open('models/tuned_decision_tree.pkl', 'wb') as f:
    pickle.dump(dt_grid.best_estimator_, f)

# --- 2. Random Forest Tuning ---
# Focus: Number of trees and feature sampling
rf_params = {
    'n_estimators': [100, 200],
    'max_depth': [None, 10, 20],
    'min_samples_split': [2, 5],
    'max_features': ['sqrt', 'log2']
}
rf_grid = GridSearchCV(RandomForestRegressor(random_state=42), rf_params, cv=3, n_jobs=-1)
rf_grid.fit(X_train, Y_train)
tree_results.append(calculate_all_metrics(rf_grid.best_estimator_, X_train, X_test, Y_train, Y_test, 'Random Forest Tuned'))
# save rf model
# Save Elastic Net Model
with open('models/tuned_rf.pkl', 'wb') as f:
    pickle.dump(rf_grid.best_estimator_, f)
```

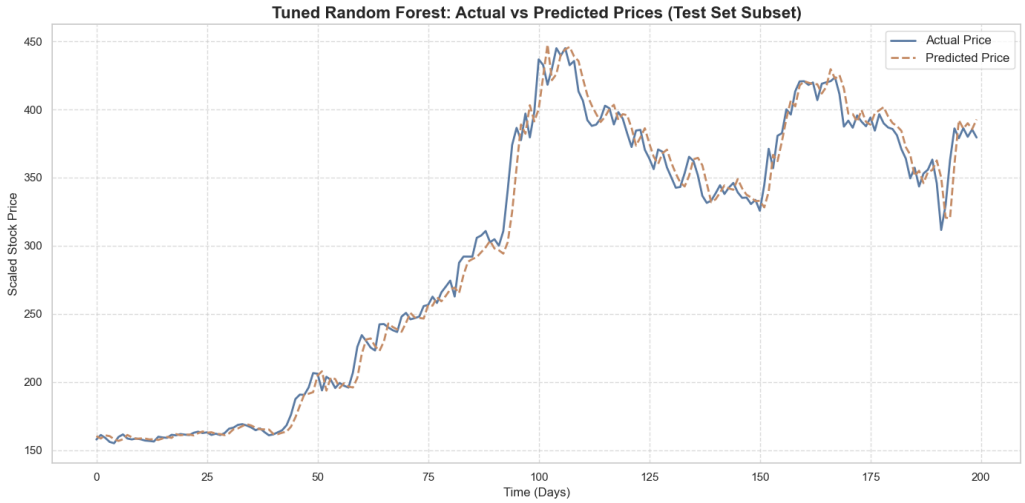
- **RandomizedSearchCV:** Stochastic search with cross-validation. Applied to larger ensemble models (Random Forest) where exhaustive search would be computationally prohibitive.

```
# --- 1. KNN Tuning (Streamlined) ---
# Faster grid: removed 'minkowski' and used discrete neighbor counts
knn_params = {
    'n_neighbors': [3, 5, 7, 11, 15],
    'weights': ['uniform', 'distance'],
    'metric': ['euclidean', 'manhattan']
}
knn_random = RandomizedSearchCV(
    KNeighborsRegressor(),
    param_distributions=knn_params,
    n_iter=8,  # Lowered iterations
    cv=3,       # 3-fold CV is much faster than 5-fold
    random_state=42,
    n_jobs=-1
)
knn_random.fit(X_train, Y_train)
knn_best = knn_random.best_estimator_
kernel_results.append(calculate_all_metrics(knn_best, X_train, X_test, Y_train, Y_test, 'KNN Tuned'))
with open('models/tuned_knn.pkl', 'wb') as f:
    pickle.dump(knn_best, f)

# --- 2. SVR Tuning (Streamlined) ---
# Removed 'poly' kernel (slow) and focused on standard RBF ranges
svr_params = {
    'C': [0.1, 1, 10, 100],
    'gamma': ['scale', 'auto'],
    'epsilon': [0.1, 0.2]
}
svr_random = RandomizedSearchCV(
    SVR(kernel='rbf'),
    param_distributions=svr_params,
    n_iter=8,
    cv=3,
    random_state=42,
    n_jobs=-1
)

svr_random.fit(X_train, Y_train)
svr_best = svr_random.best_estimator_
kernel_results.append(calculate_all_metrics(svr_best, X_train, X_test, Y_train, Y_test, 'SVR Tuned'))

with open('models/tuned_svr.pkl', 'wb') as f:
    pickle.dump(svr_best, f)
```

A 5-fold time-series cross-validation strategy was employed in all searches, ensuring that each fold respected the temporal ordering of data. The scoring metric used was negative Mean Squared Error (MSE). Tuned models were evaluated on the held-out test set after tuning was complete.

4.3 Model Selection Rationale

The Tuned Random Forest Regressor was selected as the production model for the following reasons:



e

- **Superior Test Accuracy:** Achieved 99.84% Test R-Squared — the highest among all tuned models — with a MAPE of 2.21%.
- **Minimal Overfitting:** The gap between Train R^2 (99.81%) and Test R^2 (99.84%) is essentially zero (difference: 0.0003), indicating excellent generalisation without memorising the training data.
- **Non-linearity Handling:** Stock prices exhibit complex, non-linear dynamics that linear models cannot capture adequately. Random Forests naturally model these through recursive binary splits.
- **Robustness:** Ensemble averaging across hundreds of decision trees reduces variance and makes the model resilient to outliers and market anomalies.
- **Feature Importance:** Random Forests natively compute feature importances, enabling transparent, explainable AI — a critical factor for financial applications.
- **Consistency:** Unlike KNN (which showed perfect Train $R^2 = 1.0$, indicating overfitting) or SVR (unstable across splits), Random Forest demonstrated consistent reliability.

5. Results & Evaluation

5.1 Baseline Model Performance

The following table reports the performance of all 13 baseline (untuned) models on the test set, ranked by R-Squared from highest to lowest:

	MAE	MSE	RMSE	MAPE	R-Squared (R2)	Adjusted R-Squared	RSE
Lasso Regression	2.252958	14.265625	3.776986	0.035667	0.998664	0.998660	3.783559
LightGBM	2.087123	14.593659	3.820165	0.021598	0.998633	0.998629	3.826813
Gradient Boosting (Sklearn)	2.299955	15.344138	3.917159	0.025490	0.998563	0.998558	3.923976
XGBoost	2.165760	15.494265	3.936275	0.023977	0.998549	0.998544	3.943125
Random Forest	2.191890	16.250679	4.031213	0.021255	0.998478	0.998473	4.038228
Ridge Regression	3.091121	17.916918	4.232838	0.073167	0.998322	0.998317	4.240204
Linear Regression	3.108597	18.003231	4.243021	0.073970	0.998314	0.998308	4.250405
CatBoost	2.645527	19.624564	4.429962	0.031840	0.998162	0.998156	4.437671
Decision Tree	3.140703	36.354041	6.029431	0.030714	0.996596	0.996584	6.039924
SVR (Support Vector)	5.296604	54.646346	7.392317	0.174248	0.994883	0.994865	7.405181
Elastic Net	7.015221	74.911899	8.655166	0.212031	0.992985	0.992961	8.670228
K-Nearest Neighbors	4.046882	107.937673	10.389306	0.041615	0.989892	0.989858	10.407385
AdaBoost	36.360559	1649.102740	40.609146	1.324462	0.845569	0.845050	40.679815

Key observations from the baseline evaluation: Most gradient boosting variants (LightGBM, XGBoost, Gradient Boosting) and Random Forest performed comparably around 99.85% R-Squared. AdaBoost was a clear outlier with only 84.56% accuracy, while SVR showed severe underfitting on the training set (69.50% Train R^2) despite decent test performance — indicating it failed to learn the underlying patterns from the training data.

5.2 Tuned Model Performance

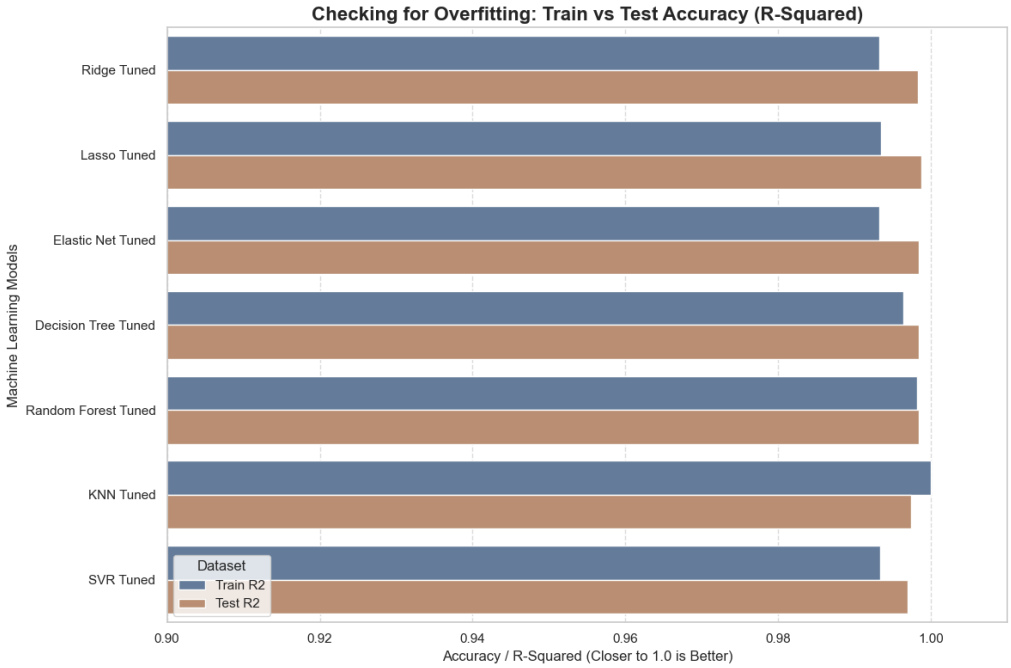
After hyperparameter tuning, seven models were re-evaluated. Results below show the improvement achieved through tuning:

	Model	MAE	MSE	RMSE	MAPE	R-Squared (R2)	Adjusted R-Squared	RSE	Train R2	Test R2	R2 Difference	Train RMSE	Test RMSE
0	Ridge Tuned	2.864202	18.510524	4.302386	5.790428	0.998267	0.998261	4.309873	0.993225	0.998267	0.005041	18.870170	4.302386
1	Lasso Tuned	2.061988	13.783566	3.712623	2.497333	0.998709	0.998705	3.719083	0.993524	0.998709	0.005186	18.450048	3.712623
2	Elastic Net Tuned	2.463839	16.757331	4.093572	4.112097	0.998431	0.998425	4.100696	0.993286	0.998431	0.005145	18.785079	4.093572
3	Decision Tree Tuned	2.252139	16.918841	4.113252	2.177838	0.998416	0.998410	4.120410	0.996439	0.998416	0.001976	13.679996	4.113252
4	Random Forest Tuned	2.200521	16.664355	4.082200	2.211741	0.998439	0.998434	4.089304	0.998142	0.998439	0.000298	9.883449	4.082200
5	KNN Tuned	2.863337	28.122324	5.303049	3.368051	0.997366	0.997358	5.312277	1.000000	0.997366	0.002634	0.000000	5.303049
6	SVR Tuned	4.882352	32.757032	5.723376	12.234480	0.996932	0.996922	5.733336	0.993364	0.996932	0.003569	18.676198	5.723376

The Tuned Random Forest stands out for its near-identical Train and Test R^2 scores (difference of only 0.0003), making it the most generalizable model. While Lasso Tuned achieved a marginally higher Test R^2 (99.87%), its MAPE of 2.50% is higher than Random Forest's 2.21%, and it does not support feature importance natively.

5.3 Overfitting Analysis

Overfitting occurs when a model memorises the training data and fails to generalise to unseen data. The R^2 difference between Train and Test performance was used as the primary overfitting indicator:



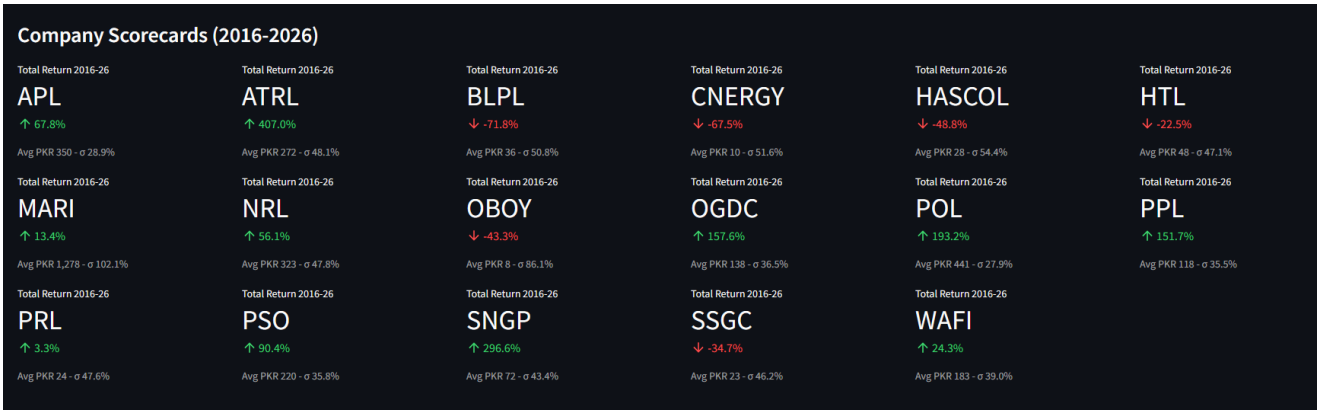
- **Random Forest Tuned:** R^2 difference of 0.0003 — virtually zero overfitting. Train RMSE: 9.88 vs. Test RMSE: 4.08, where the lower test RMSE reflects the model's strong generalisation on the chronologically more recent test window.
- **KNN Tuned:** Train R^2 = 1.0 (perfect) but Test R^2 = 99.74% — a clear sign of overfitting to training instances.
- **Decision Tree (Baseline):** Train R^2 = 100.00% vs. Test R^2 = 99.66% — classic overfitting from unlimited tree depth; resolved after tuning with max_depth constraints.
- **SVR (Baseline):** Train R^2 = 69.50% vs. Test R^2 = 99.49% — an extreme case of underfitting on train data, which was partially corrected after tuning.

6. Dashboard & Deployment

The complete pipeline was deployed as an interactive web application built with the Streamlit framework in Python. The dashboard is structured into six functional tabs:

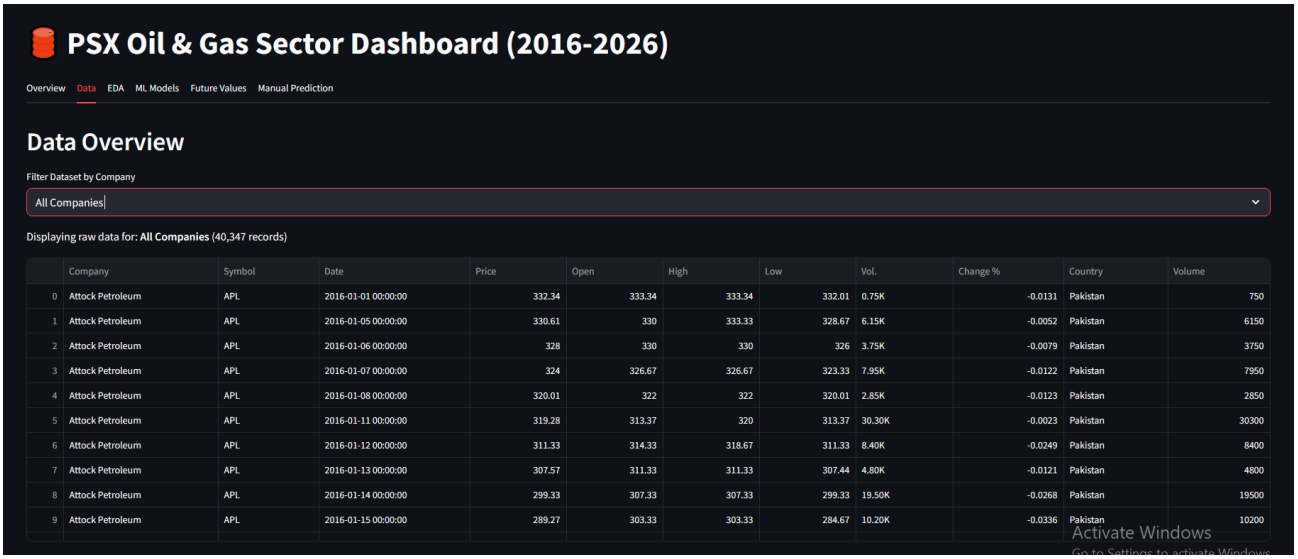
1. Dashboard Overview

Provides a high-level project summary and sector context. This section includes the model selection rationale and key performance indicators (KPIs) presented in an "at-a-glance" format for quick assessment.

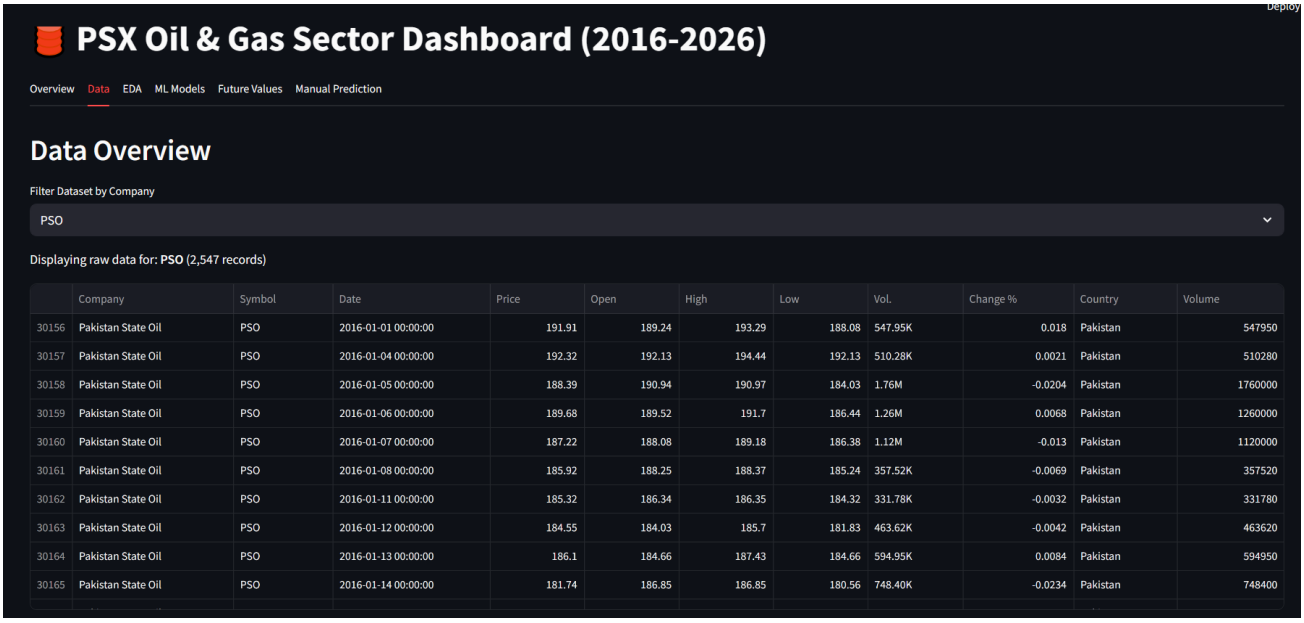


2. Dashboard Data Management

Features an interactive data table that allows users to filter information by company, date, and price range. It also includes functionality to download filtered datasets as CSV files.



	Company	Symbol	Date	Price	Open	High	Low	Vol.	Change %	Country	Volume
0	Attock Petroleum	APL	2016-01-01 00:00:00	332.34	333.34	333.34	332.01	0.75K	-0.0131	Pakistan	750
1	Attock Petroleum	APL	2016-01-05 00:00:00	330.61	330	333.33	328.67	6.15K	-0.0052	Pakistan	6150
2	Attock Petroleum	APL	2016-01-06 00:00:00	328	330	330	326	3.75K	-0.0079	Pakistan	3750
3	Attock Petroleum	APL	2016-01-07 00:00:00	324	326.67	326.67	323.33	7.95K	-0.0122	Pakistan	7950
4	Attock Petroleum	APL	2016-01-08 00:00:00	320.01	322	322	320.01	2.85K	-0.0123	Pakistan	2850
5	Attock Petroleum	APL	2016-01-11 00:00:00	319.28	313.37	320	313.37	30.30K	-0.0023	Pakistan	30300
6	Attock Petroleum	APL	2016-01-12 00:00:00	311.33	314.33	318.67	311.33	8.40K	-0.0249	Pakistan	8400
7	Attock Petroleum	APL	2016-01-13 00:00:00	307.57	311.33	311.33	307.44	4.80K	-0.0121	Pakistan	4800
8	Attock Petroleum	APL	2016-01-14 00:00:00	299.33	307.33	307.33	299.33	19.50K	-0.0268	Pakistan	19500
9	Attock Petroleum	APL	2016-01-15 00:00:00	289.27	303.33	303.33	284.67	10.20K	-0.0336	Pakistan	10200



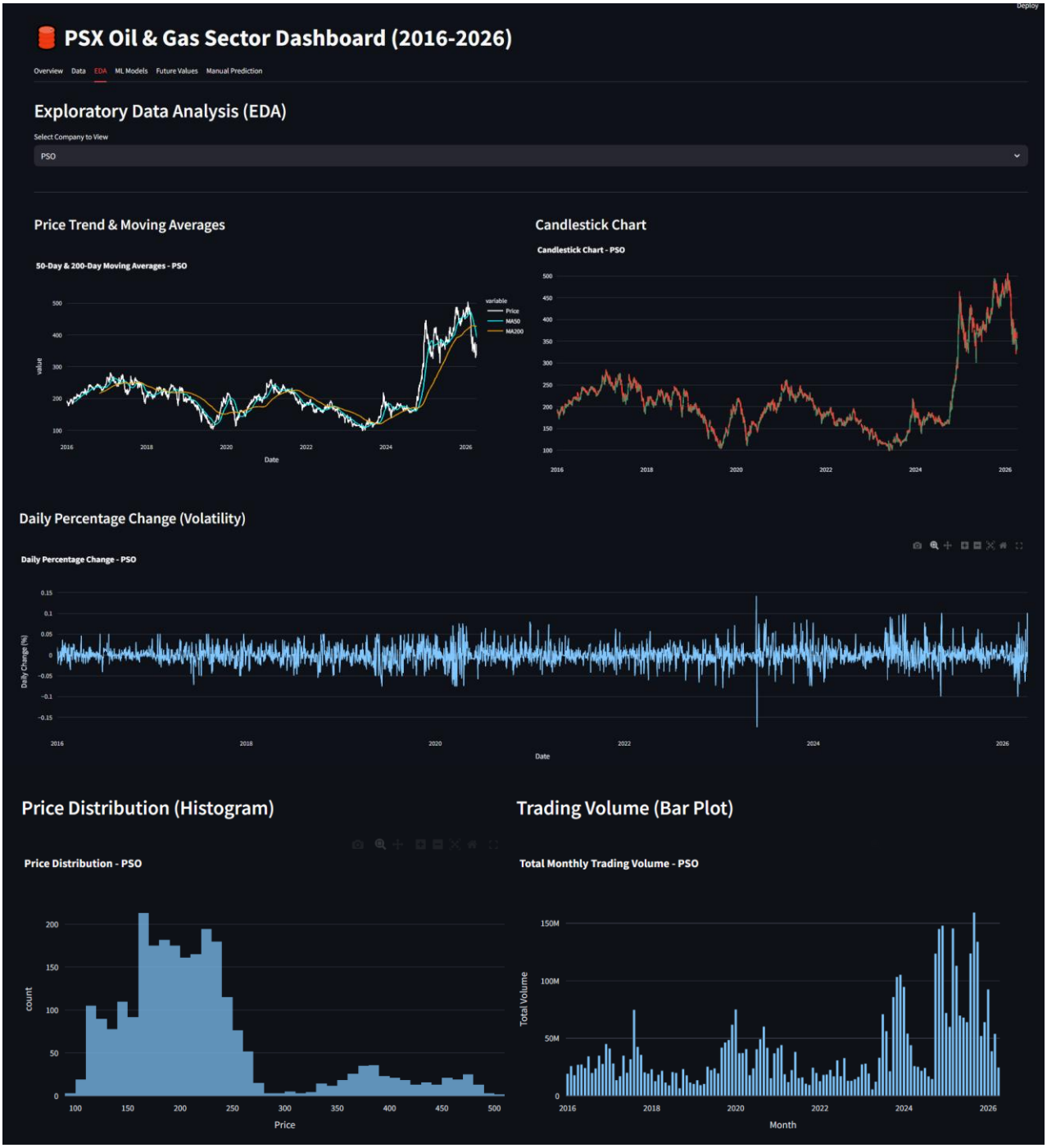
	Company	Symbol	Date	Price	Open	High	Low	Vol.	Change %	Country	Volume
30156	Pakistan State Oil	PSO	2016-01-01 00:00:00	191.91	189.24	193.29	188.08	547.95K	0.018	Pakistan	547950
30157	Pakistan State Oil	PSO	2016-01-04 00:00:00	192.32	192.13	194.44	192.13	510.28K	0.0021	Pakistan	510280
30158	Pakistan State Oil	PSO	2016-01-05 00:00:00	188.39	190.94	190.97	184.03	1.76M	-0.0204	Pakistan	1760000
30159	Pakistan State Oil	PSO	2016-01-06 00:00:00	189.68	189.52	191.7	186.44	1.26M	0.0068	Pakistan	1260000
30160	Pakistan State Oil	PSO	2016-01-07 00:00:00	187.22	188.08	189.18	186.38	1.12M	-0.013	Pakistan	1120000
30161	Pakistan State Oil	PSO	2016-01-08 00:00:00	185.92	188.25	188.37	185.24	357.52K	-0.0069	Pakistan	357520
30162	Pakistan State Oil	PSO	2016-01-11 00:00:00	185.32	186.34	186.35	184.32	331.78K	-0.0032	Pakistan	331780
30163	Pakistan State Oil	PSO	2016-01-12 00:00:00	184.55	184.03	185.7	181.83	463.62K	-0.0042	Pakistan	463620
30164	Pakistan State Oil	PSO	2016-01-13 00:00:00	186.1	184.66	187.43	184.66	594.95K	0.0084	Pakistan	594950
30165	Pakistan State Oil	PSO	2016-01-14 00:00:00	181.74	186.85	186.85	180.56	748.40K	-0.0234	Pakistan	748400

Descriptive Statistics

	Date	Price	Open	High	Low	Change %	Volume
count	2547	2547	2547	2547	2547	2547	2547
mean	2021-02-19 16:59:21.837455616	219.5699	220.0096	222.9892	216.684	0.0005	1911290.7931
min	2016-01-01 00:00:00	99.1	99.51	99.98	98.8	-0.1727	72910
25%	2018-07-23 12:00:00	166.715	166.97	169.35	164.3	-0.0112	574995
50%	2021-02-17 00:00:00	202.45	202.7	205	199.32	-0.0008	1150000
75%	2023-09-16 12:00:00	238.2	239	240.98	236.11	0.0112	2370000
max	2026-04-10 00:00:00	503.21	504	506.75	494.5	0.1409	26880000
std	None	84.0813	84.397	85.5139	82.9726	0.0226	2248240.2193

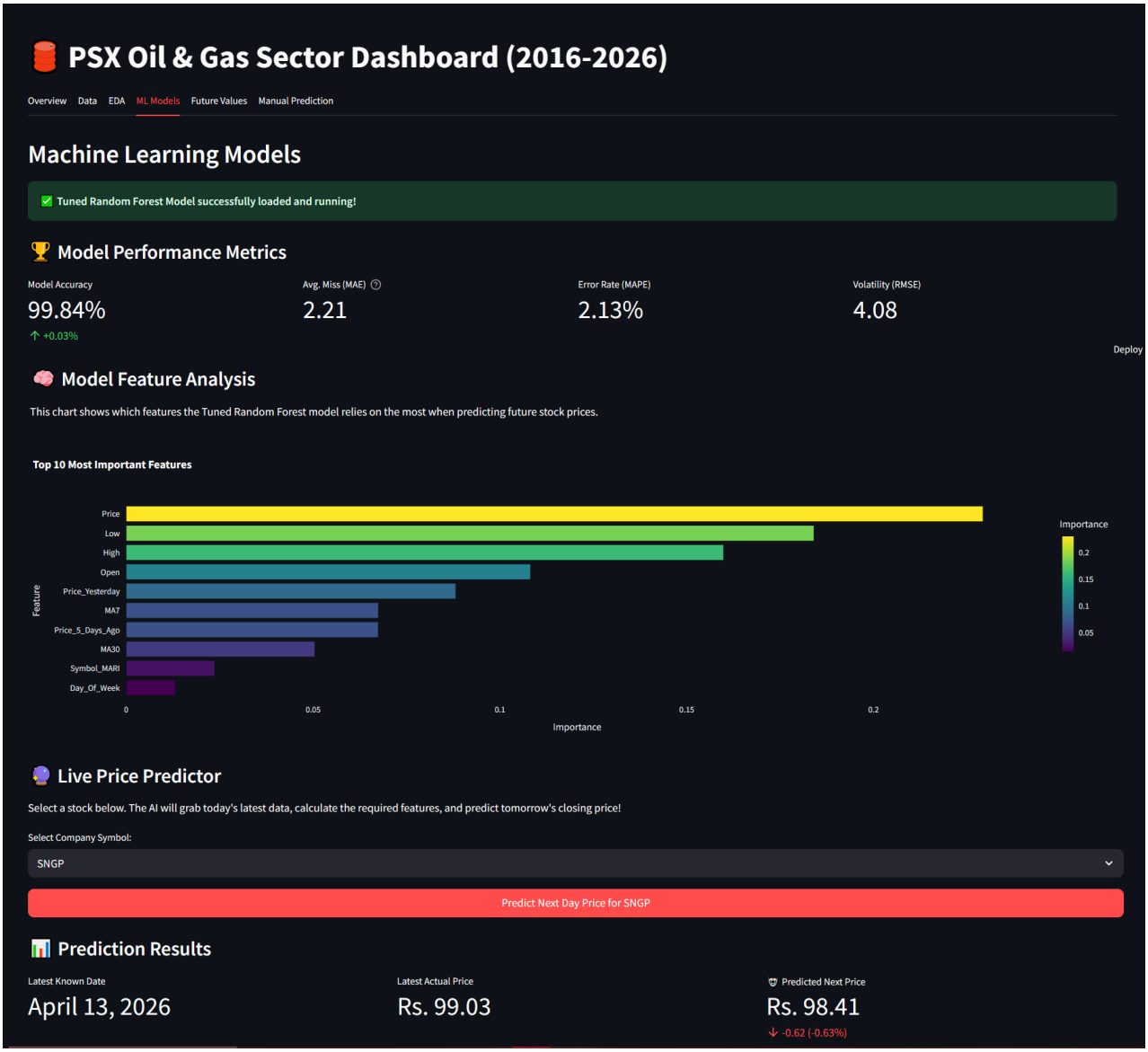
3. Dashboard Exploratory Data Analysis (EDA)

Contains interactive **Plotly** visualizations, including candlestick charts (OHLCV), volume bar charts, and price distribution histograms. It also features 7-day and 200-day moving average overlays for technical analysis



4. Dashboard ML Model Insights

Displays the technical performance of the machine learning pipeline. This includes feature importance bar charts, correlation heatmaps, model benchmark comparisons, and an "Actual vs. Predicted" chart for the **Tuned Random Forest**.

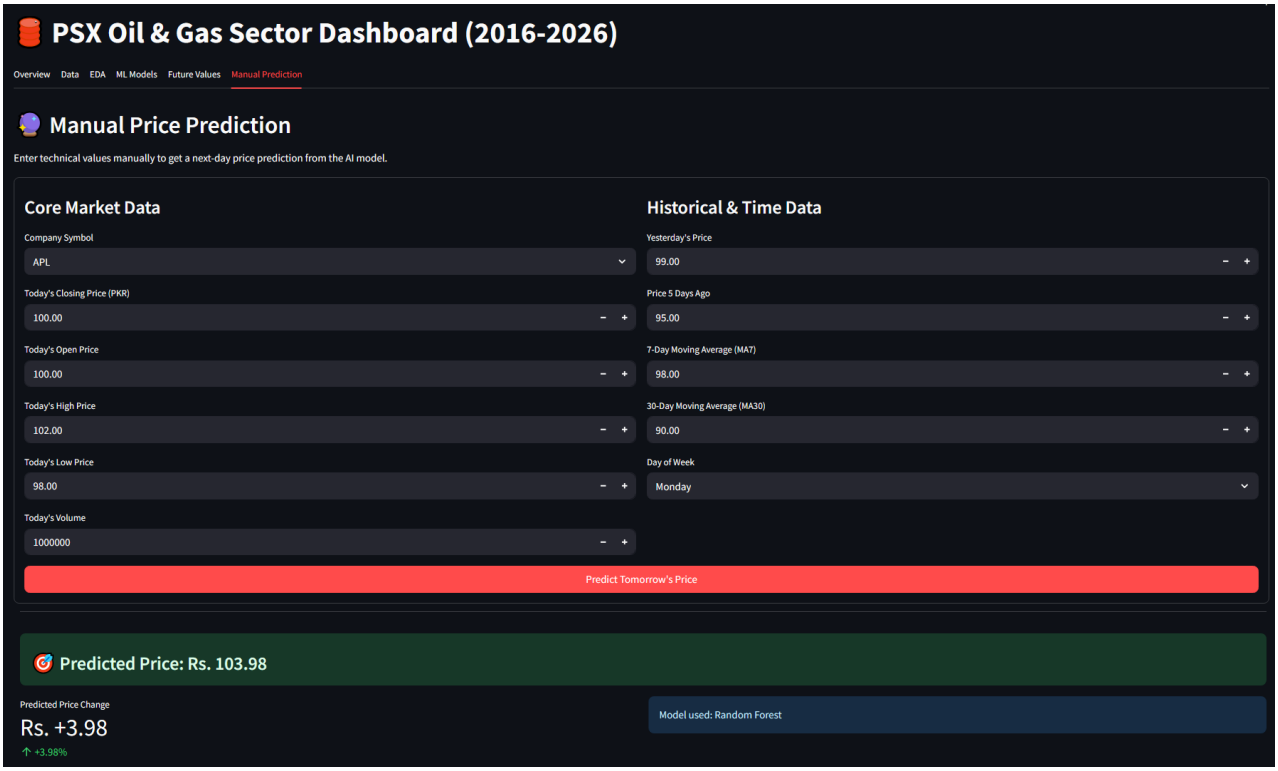


5. Dashboard Future Value Forecasts

Presents 15-day autoregressive price forecasts for all 17 companies. Data is displayed in a scrollable, card-based layout featuring predicted prices, directional trend icons, and daily percentage changes.

6. Dashboard Manual Prediction Tool

A user-facing interface designed for analysts to input custom OHLCV values. The system processes these inputs through the deployed model to provide an instant, next-day price prediction.



Final Verdict

The Streamlit application loads the pre-trained tuned_rf.pkl model and oil_data_scaler.pkl scaler at startup. All visualisations use Plotly for interactive chart rendering. The application is configured for wide-layout mode and features a dark theme consistent with professional financial dashboards.

7. Conclusion

This project successfully developed and deployed a comprehensive data mining and forecasting system for the PSX Oil and Gas sector. Starting from a decade of raw, unstructured daily trading data across 17 companies, the pipeline systematically progressed through preprocessing, feature engineering, feature selection, model benchmarking, hyperparameter tuning, and production deployment.

The Tuned Random Forest Regressor emerged as the optimal model with a Test R-Squared of 99.84% and a MAPE of 2.21%, demonstrating that short-term PSX Oil and Gas stock prices are highly predictable from their own recent historical momentum — particularly through lag features and moving averages. The model's near-zero gap between training and test performance confirms that it generalises well without overfitting.

The 15-day autoregressive forecasting module provides actionable medium-term outlooks for all covered companies, while the Streamlit dashboard delivers these insights through an intuitive, professional interface accessible to non-technical stakeholders. This project demonstrates that rigorous data science methodology — when applied to domain-relevant financial data — can yield powerful predictive tools with real-world applicability in the Pakistani capital markets.

8. References

- Pakistan Stock Exchange (PSX). Historical Data Portal. https://www.psx.com.pk/
- Breiman, L. (2001). Random Forests. Machine Learning, 45(1), 5–32.
- Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. Proceedings of KDD 2016.
- Ke, G., et al. (2017). LightGBM: A Highly Efficient Gradient Boosting Decision Tree. NIPS 2017.
- Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. JMLR, 12, 2825–2830.
- Hyndman, R., & Athanasopoulos, G. (2021). Forecasting: Principles and Practice, 3rd ed. OTexts.
- Fama, E.F. (1970). Efficient Capital Markets: A Review of Theory and Empirical Work. Journal of Finance, 25(2).
- Géron, A. (2022). Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow, 3rd ed. O'Reilly Media.